Звіт з практичної роботи №10

Робота з файлами: збереження і аналіз даних

Студент:	Nikita Krilovets
Група:	IT-31
Рік народження (year):	2009
Мова (language):	Python
Витрати:	expenses.txt (13 записів)
Каталог:	practice22

Мета роботи

Навчитися будувати шляхи незалежно від місця запуску програми, створювати каталоги й файли, зберігати дані між запусками, розбирати текстовий файл на структуровані записи, формувати звіт та працювати з аргументами командного рядка. Усі завдання виконано з особистими даними: ім'я Nikita, прізвище Krilovets, група IT-31, рік народження 2009, улюблена мова — Python. Проєкт розміщено в каталозі **practice22**, кожне завдання — в окремому файлі (**task1.py**, **task2.py**, **task3.py**).

Завдання 1. Робоче місце програми (task1.py)

Пункт 1. Виведення чотирьох шляхів

За допомогою модуля **pathlib** програма отримує поточний робочий каталог (**Path.cwd()**), абсолютний шлях до власного файлу (**Path(__file__).resolve()**), каталог скрипта (**.parent**) та домашній каталог (**Path.home()**) і виводить їх із підписами.

task1.py

```python
#Пункт 1
from pathlib import Path

cwd = Path.cwd()
script = Path(__file__).resolve()
script_dir = script.parent
home = Path.home()

print("cwd:", cwd)
print("script:", script)
print("script dir:", script_dir)
print("home:", home)
```

Рис. 1 — Завдання 1, пункт 1: код програми

```
cwd: C:\Users\admin\Desktop\Практичні з програмування\practice22
script: C:\Users\admin\Desktop\Практичні з програмування\practice22\task1.py
script dir: C:\Users\admin\Desktop\Практичні з програмування\practice22
home: C:\Users\admin
```

Рис. 2 — Завдання 1, пункт 1: результат виконання

Пункт 2. Створення каталогів data, reports, backups

Шляхи до каталогів будуються від каталогу скрипта через оператор **/**. Каталоги створюються методом **mkdir(exist_ok=True)**, тому повторний запуск програми не викликає помилки. У провіднику видно створені каталоги backups, data і reports.

```python

#Пункт 2
data_dir = script_dir / "data"
reports_dir = script_dir / "reports"
backups_dir = script_dir / "backups"

data_dir.mkdir(exist_ok=True)
reports_dir.mkdir(exist_ok=True)
backups_dir.mkdir(exist_ok=True)
```

Рис. 3 — Завдання 1, пункт 2: код створення каталогів

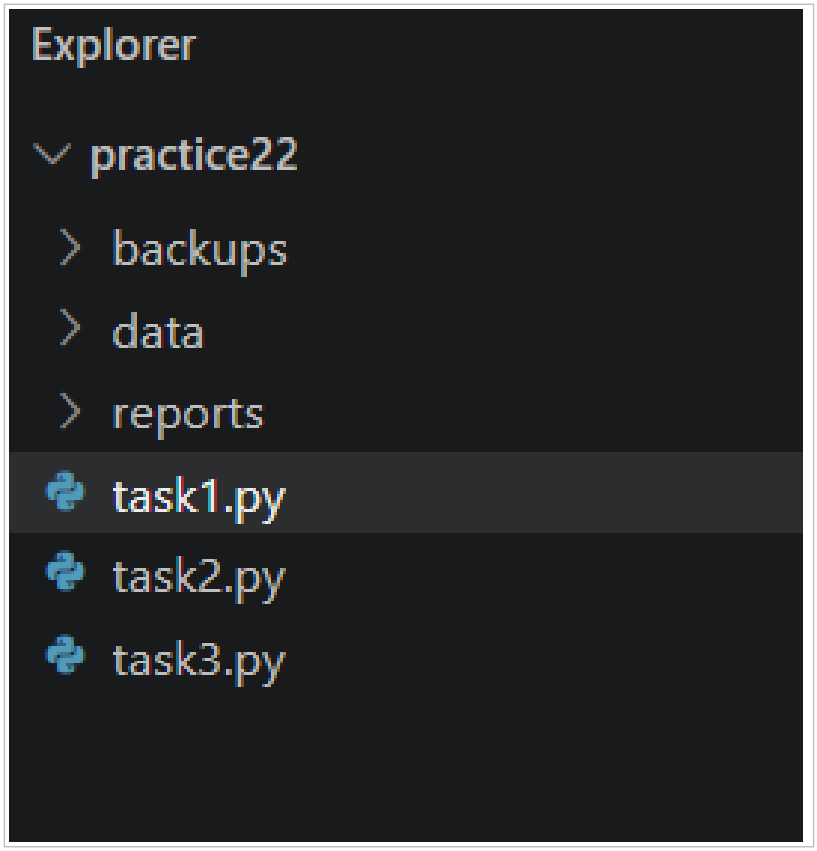


Рис. 4 — Завдання 1, пункт 2: структура проєкту в Explorer після створення каталогів

Пункт 3. Запис файлу data/profile.txt

У файл **data/profile.txt** записано п'ять рядків у форматі *key: value* з особистими даними: name, surname, group, year, language. Файл відкривається в режимі запису з кодуванням UTF-8.

```
22
23  #Пункт 3
24  profile_path = data_dir / "profile.txt"
25
26  with profile_path.open("w", encoding="utf-8") as file:
27      file.write("name: Nikita\n")
28      file.write("surname: Krilovets\n")
29      file.write("group: IT-31\n")
30      file.write("year: 2009\n")
31      file.write("language: Python\n")
```

Рис. 5 — Завдання 1, пункт 3: код запису profile.txt

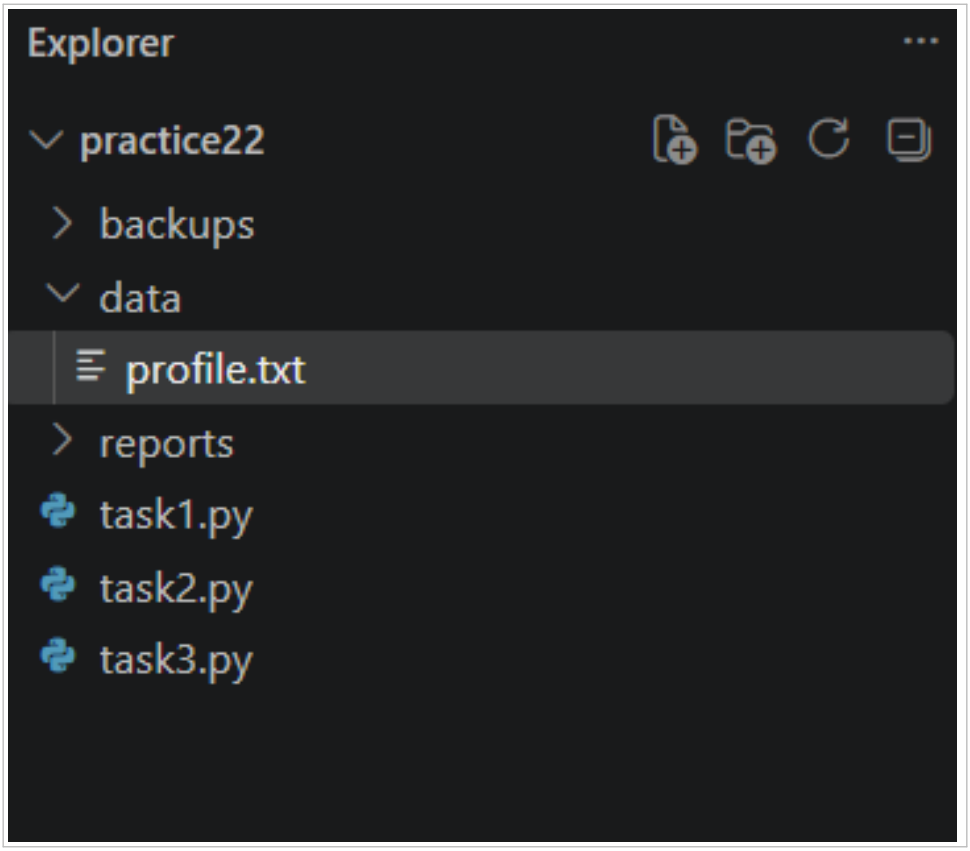


Рис. 6 — Завдання 1, пункт 3: файл profile.txt з'явився в каталозі data

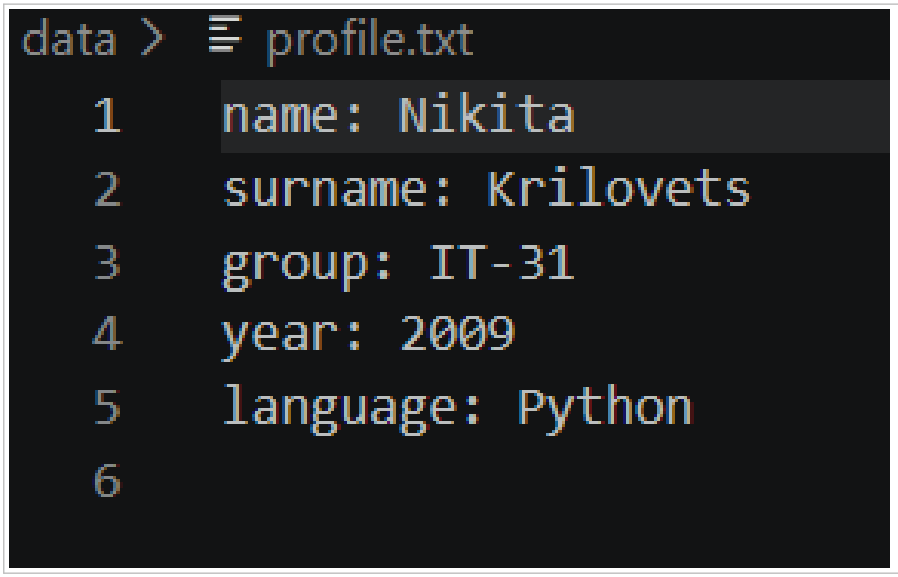


Рис. 7 — Завдання 1, пункт 3: вміст файлу profile.txt

Пункт 4. Властивості файлу

Для файлу profile.txt виведено: абсолютний шлях (**resolve()**), шлях відносно каталогу скрипта (**relative_to**), ім'я (**name**), ім'я без розширення (**stem**), розширення (**suffix**), ім'я батьківського каталогу (**parent.name**) та розмір у байтах (**stat().st_size**) — 78 байт.

```python

#Пункт 4
print("profile path:", profile_path.resolve())
print("relative:", profile_path.relative_to(script_dir))
print("name:", profile_path.name)
print("stem:", profile_path.stem)
print("suffix:", profile_path.suffix)
print("parent:", profile_path.parent.name)
print("size:", profile_path.stat().st_size, "bytes")
```

Рис. 8 — Завдання 1, пункт 4: код виведення властивостей файлу

```
script: C:\Users\admin\Desktop\Практичні з програмування\practice22\task1.py
script dir: C:\Users\admin\Desktop\Практичні з програмування\practice22
home: C:\Users\admin
profile path: C:\Users\admin\Desktop\Практичні з програмування\practice22\data\profile.txt
relative: data\profile.txt
name: profile.txt
stem: profile
suffix: .txt
parent: data
size: 78 bytes
```

Рис. 9 — Завдання 1, пункт 4: результат виконання

Пункт 5. Дерево проєкту

Для виведення дерева використано рекурсивний обхід **rglob("*")** з сортуванням. Відступ залежить від кількості частин відносного шляху; для каталогів додається символ /.

```python

#Пункт 5
print("\nproject tree:")

for path in sorted(script_dir.rglob("*")):
    relative = path.relative_to(script_dir)
    indent = "  " * (len(relative.parts) - 1)

    if path.is_dir():
        print(f"{indent}{path.name}/")
    else:
        print(f"{indent}{path.name}")
```

Рис. 10 — Завдання 1, пункт 5: код виведення дерева проєкту

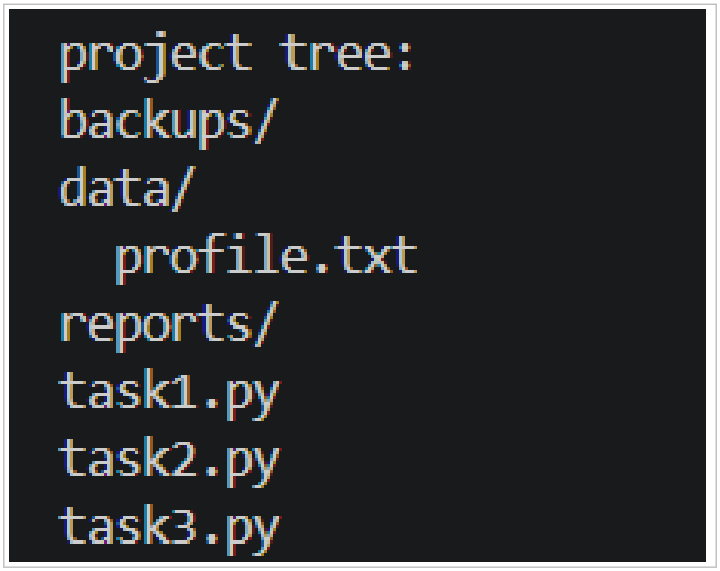


Рис. 11 — Завдання 1, пункт 5: результат — дерево проєкту

Завдання 2. Пам'ять між запусками (task2.py)

Програма складається з функцій та одного виклику **main()** наприкінці файлу. Дані про запуски зберігаються у файлі **data/runs.log**; кожен запис має вигляд *номер;дата і час;ім'я*.

Пункти 1–3. Функції count_runs, add_run, show_history

Пункт 1 — count_runs(path): повертає 0, якщо файлу немає, інакше — кількість непорожніх рядків. **Пункт 2 — add_run(path, number):** відкриває файл у режимі дописування (**"a"**) і додає один рядок з номером запуску, поточним часом (**datetime.now().strftime**) та ім'ям, нічого не стираючи. **Пункт 3 — show_history(path, last=3):** виводить загальну кількість запусків, три останні записи та час першого запуску.

task2.py

```
2   from pathlib import Path
3   from datetime import datetime
4
5
6   def count_runs(path):
7       if not path.exists():
8           return 0
9
10      with path.open("r", encoding="utf-8") as file:
11          lines = file.readlines()
12
13      return len([line for line in lines if line.strip()])
14
15
16  # Пункт 2
17  def add_run(path, number):
18      stamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
19
20      with path.open("a", encoding="utf-8") as file:
21          file.write(f"{number};{stamp};Nikita Krilovets\n")
22
23  #Пункт 3
24  def show_history(path, last=3):
25      with path.open("r", encoding="utf-8") as file:
26          lines = [line.strip() for line in file if line.strip()]
27
28      print("runs so far:", len(lines))
29      print("last runs:")
30
31      for line in lines[-last:]:
32          number, stamp, name = line.split(";")
33          print(number, stamp, name)
34
35      first = lines[0].split(";")
36      print("first run was at:", first[1])
```

Рис. 12 — Завдання 2, пункти 1–3: код функцій count_runs, add_run, show_history

Пункт 4. Функція main()

У **main()** визначається шлях до **data/runs.log**, обчислюється номер поточного запуску, додається запис, виводиться привітання з ім'ям та показується історія. При першому запуску додатково повідомляється, що файл щойно створено.

```python
#Пункт 4
def main():
    path = Path(__file__).resolve().parent / "data" / "runs.log"

    runs = count_runs(path)
    run_number = runs + 1

    add_run(path, run_number)

    print(f"Hello, Nikita Krilovets! This is run number {run_number}.")

    if run_number == 1:
        print("runs.log was just created.")

    show_history(path)

main()
```

Рис. 13 — Завдання 2, пункт 4: код функції main() і виклик main()

```
Hello, Nikita Krilovets! This is run number 5.
runs so far: 5
last runs:
3 2026-10-08 12:49:56 Nikita Krilovets
4 2026-10-08 12:49:59 Nikita Krilovets
5 2026-10-08 12:50:00 Nikita Krilovets
first run was at: 2026-10-08 12:49:47
```

Рис. 14 — Завдання 2, пункт 4: результат п'ятого запуску програми

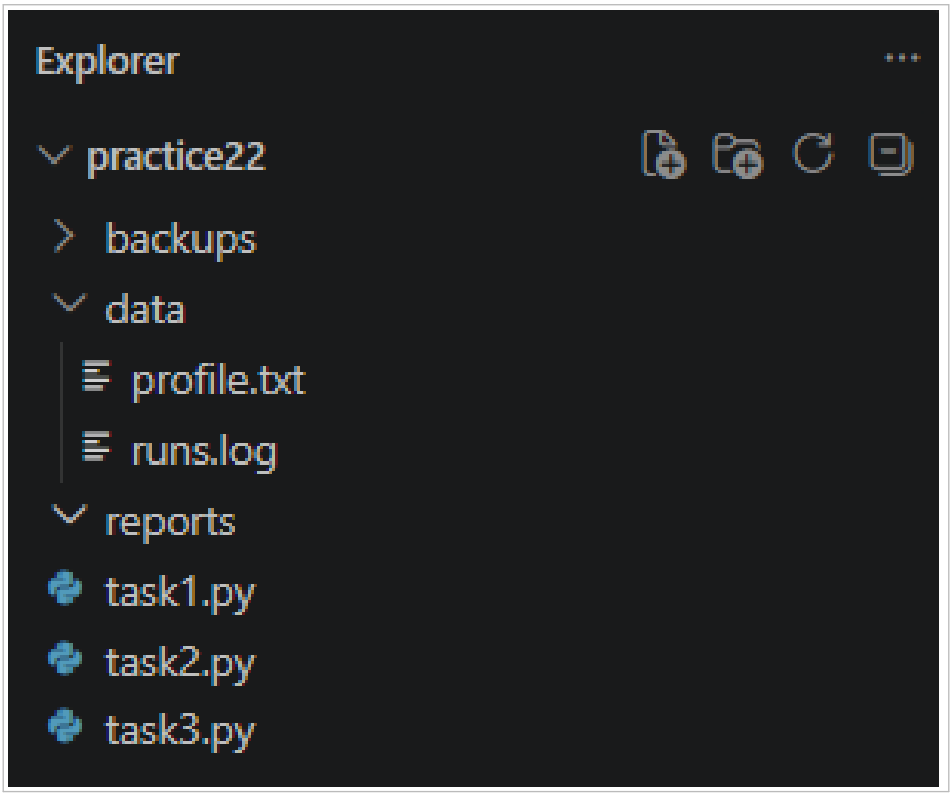


Рис. 15 — Завдання 2, пункт 4: у каталозі data з'явився файл runs.log

Завдання 3. Трекер витрат (task3.py)

Консольна утиліта для обліку витрат із трьома командами: **add**, **list**, **report**. Команда та її аргументи беруться з **sys.argv**. Програму розроблено поетапно.

Пункт 1. Розбір команд із командного рядка

Команда читається з **sys.argv[1]**, далі конструкція **if / elif / else** вибирає дію: **add**, **list**, **report** або повідомлення про невідому команду. На цьому етапі гілки лише виводять текстові заглушки.

```
PS C:\Users\admin\Desktop\Практичні з програмування\practice22> python task3.py add
add
```

Рис. 16 — Завдання 3, пункт 1: запуск програми з командою add

task3.py

```python
import sys

command = sys.argv[1]

if command == "add":
    print("Додаємо витрату")
elif command == "list":
    print("Показуємо витрати")
elif command == "report":
    print("Показуємо звіт")
else:
    print("Невідома команда")
```

Рис. 17 — Завдання 3, пункт 1: код розбору команд

```
PS C:\Users\admin\Desktop\Практичні з програмування\practice22> python task3.py add
Додаємо витрату
```

Рис. 18 — Завдання 3, пункт 1: результат виконання команди add

Пункт 2. Команда add — запис витрати у файл

Функція **add_expense(category, amount)** дописує в кінець файлу **expenses.txt** рядок *категорія;сума* (режим **"a"**), не перечитуючи й не переписуючи файл. Категорія та сума беруться з аргументів **sys.argv[2]** і **sys.argv[3]**.



Рис. 19 — Завдання 3, пункт 2: код з функцією add_expense (початкова версія)

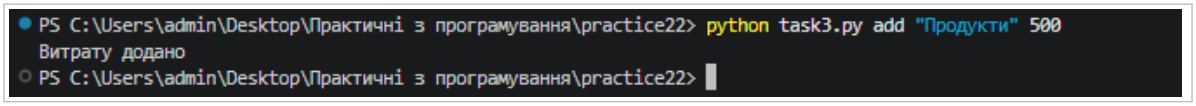


Рис. 20 — Завдання 3, пункт 2: додавання витрати «Продукти» 500

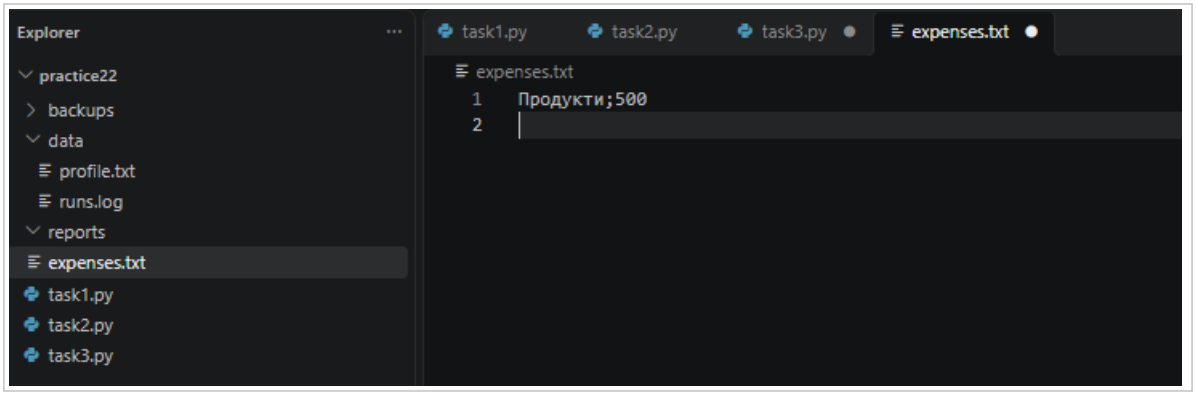


Рис. 21 — Завдання 3, пункт 2: файл expenses.txt із записом «Продукти;500»

Пункт 3. Команда list — перегляд витрат

Функція **list_expenses()** читає файл рядок за рядком і виводить кожен запис на екран.

```python

def list_expenses():
    with open("expenses.txt", "r", encoding="utf-8") as file:
        for line in file:
            print(line.strip())


command = sys.argv[1]

if command == "add":
    category = sys.argv[2]
    amount = sys.argv[3]

    add_expense(category, amount)
    print("Витрату додано")

elif command == "list":
    list_expenses()

elif command == "report":
    print("Показуємо звіт")

else:
    print("Невідома команда")
```

Рис. 22 — Завдання 3, пункт 3: код функції list_expenses

```
PS C:\Users\admin\Desktop\Практичні з програмування\practice22> python task3.py add "Транспорт" 100
Витрату додано
PS C:\Users\admin\Desktop\Практичні з програмування\practice22> python task3.py list
Продукти;500
Транспорт;100
```

Рис. 23 — Завдання 3, пункт 3: додавання витрати «Транспорт» 100 та виконання команди list

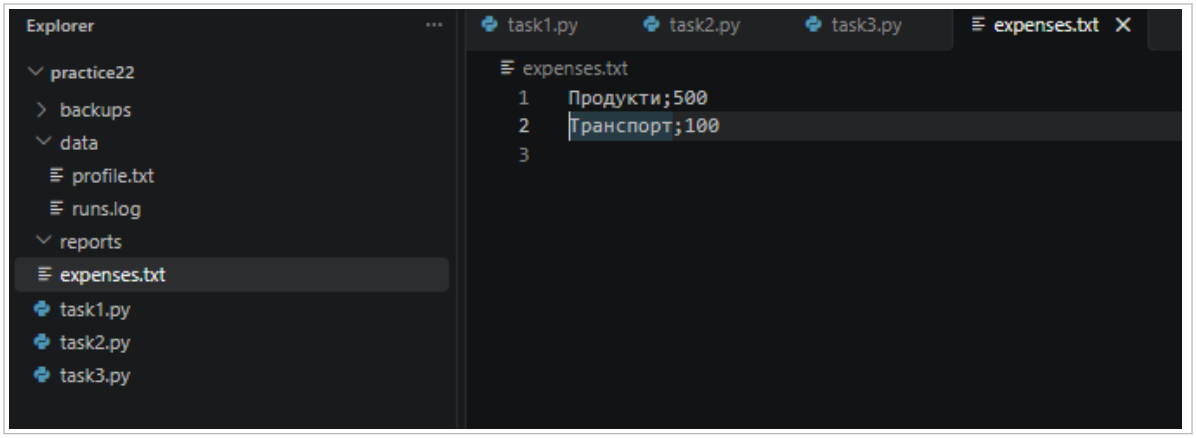


Рис. 24 — Завдання 3, пункт 3: вміст файлу expenses.txt після двох записів

Пункт 4. Команда report, обробка помилок і наповнення даними

4.1. Команда report. Функція **report_expenses()** проходить по всіх рядках файлу, розбиває кожен рядок за символом **;**, перетворює суму на число (**float**) і підсумовує. У гілці **report** викликається ця функція.

```python

def report_expenses():
    total = 0

    with open("expenses.txt", "r", encoding="utf-8") as file:
        for line in file:
            category, amount = line.strip().split(";")
            total += float(amount)

    print("Загальна сума:", total)


```

Рис. 25 — Завдання 3, пункт 4.1: код функції report_expenses

```python
    elif command == "report":
        report_expenses()

```

Рис. 26 — Завдання 3, пункт 4.1: виклик report_expenses() у гілці report

```
PS C:\Users\admin\Desktop\Практичні з програмування\practice22> python task3.py report
Загальна сума: 600.0
```

Рис. 27 — Завдання 3, пункт 4.1: результат команди report (загальна сума 600.0)

4.2. Підказка замість помилки. Якщо аргументів немає або для **add** їх замало, програма виводить підказку (usage) і завершується через **sys.exit()**, а не падає з **IndexError**.

```python

if len(sys.argv) < 2:
    print("Usage: python task3.py add/list/report")
    sys.exit()

command = sys.argv[1]

if command == "add":
    if len(sys.argv) < 4:
        print("Usage: python task3.py add \"категорія\" сума")
        sys.exit()

    category = sys.argv[2]
    amount = sys.argv[3]

    add_expense(category, amount)
    print("Витрату додано")

```

Рис. 28 — Завдання 3, пункт 4.2: код перевірки кількості аргументів

```
PS C:\Users\admin\Desktop\Практичні з програмування\practice22> python task3.py
Usage: python task3.py add/list/report
PS C:\Users\admin\Desktop\Практичні з програмування\practice22>
```

Рис. 29 — Завдання 3, пункт 4.2: запуск без аргументів — виводиться підказка

```
PS C:\Users\admin\Desktop\Практичні з програмування\practice22> python task3.py add
Usage: python task3.py add "категорія" сума
```

Рис. 30 — Завдання 3, пункт 4.2: команда add без категорії та суми — виводиться підказка

4.3. Наповнення даними, list і report. Додано ще 11 витрат командою **add** (разом 13 записів), після чого виконано команди **list** і **report**. Загальна сума витрат — 2330.0.

```
PS C:\Users\admin\Desktop\Практичні з програмування\practice22> python task3.py add "Транспорт" 100
Витрату додано
PS C:\Users\admin\Desktop\Практичні з програмування\practice22> python task3.py add "Обід" 180
Витрату додано
PS C:\Users\admin\Desktop\Практичні з програмування\practice22> python task3.py add "Мобільний зв'язок" 250
Витрату додано
PS C:\Users\admin\Desktop\Практичні з програмування\practice22> python task3.py add "Канцелярія" 120
Витрату додано
PS C:\Users\admin\Desktop\Практичні з програмування\practice22> python task3.py add "Вода" 30
Витрату додано
PS C:\Users\admin\Desktop\Практичні з програмування\practice22> python task3.py add "Перекус" 70
Витрату додано
PS C:\Users\admin\Desktop\Практичні з програмування\practice22> python task3.py add "Проїзд" 50
Витрату додано
PS C:\Users\admin\Desktop\Практичні з програмування\practice22> python task3.py add "Фільм" 200
Витрату додано
PS C:\Users\admin\Desktop\Практичні з програмування\practice22> python task3.py add "Книга" 350
Витрату додано
PS C:\Users\admin\Desktop\Практичні з програмування\practice22> python task3.py add "Побутові товари" 300
Витрату додано
PS C:\Users\admin\Desktop\Практичні з програмування\practice22> python task3.py add "Кава" 80
Витрату додано
PS C:\Users\admin\Desktop\Практичні з програмування\practice22>
```

Рис. 31 — Завдання 3, пункт 4.3: додавання витрат командою add

```
PS C:\Users\admin\Desktop\Практичні з програмування\practice22> python task3.py list
Продукти;500
Транспорт;100
Транспорт;100
Обід;180
Мобільний зв'язок;250
Канцелярія;120
Вода;30
Перекус;70
Проїзд;50
Фільм;200
Книга;350
Побутові товари;300
Кава;80
PS C:\Users\admin\Desktop\Практичні з програмування\practice22> python task3.py report
Загальна сума: 2330.0
```

Рис. 32 — Завдання 3, пункт 4.3: результат команд list і report (13 записів, сума 2330.0)

Пункт 5. Ігнорування порожніх рядків і коментарів

Щоб програма не ламалася на «брудному» файлі, у функціях **list_expenses()** і **report_expenses()** кожен рядок очищується методом **strip()**, а порожні рядки та рядки, що починаються з **#**, пропускаються оператором **continue**.



```
8
9   def list_expenses():
10      with open("expenses.txt", "r", encoding="utf-8") as file:
11          for line in file:
12              line = line.strip()
13
14              if not line or line.startswith("#"):
15                  continue
16
17              print(line)
18
19
20  def report_expenses():
21      total = 0
22
23      with open("expenses.txt", "r", encoding="utf-8") as file:
24          for line in file:
25              line = line.strip()
26
27              if not line or line.startswith("#"):
28                  continue
29
30              category, amount = line.split(";")
31              total += float(amount)
32
33      print("Загальна сума:", total)
34
```

Рис. 33 — Завдання 3, пункт 5: код, що ігнорує порожні рядки та рядки-коментарі